Design principles, patterns, and backend architecture

9 MAIN SECTIONS • PRINT-READY EDITION

In this guide

1. How to study this guide 2
2. Design principles you should know 2
3. Object-level design patterns 12
4. Architecture patterns and when to choose them 23
5. Worked design: submit a commercial payment 34
6. Fast comparisons interviewers ask 37
7. Practice questions and model answers 40
8. Four worked demonstrations: prove the choice, not just the name 42
9. Final one-page recall 45

Senior Software Engineer interview study guide | October 22, 2026

Expanded edition (October 23, 2026): Each topic now includes a practice case, a reason for the choice, and evidence or a counterexample. All new scenarios are hypothetical.

This guide builds on your technical and coding preparation. It uses commercial payments, reconciliation, identity, and user-management examples because those appear in your resumes. The examples are **practice designs**, not descriptions of American Express systems. State what you actually implemented when discussing your own work.

Code examples: The code anchors are intentionally short teaching fragments, not complete applications. Some use illustrative types (such as `Money`, `Payment`, or repositories) and omit imports, wiring, error mapping, and security. Some callback/lifecycle sketches are marked pseudocode. They demonstrate responsibility and failure behavior; they are **not drop-in production payment code**. Follow each with its verification question.

Print layout: Long comparisons and failure matrices use vertical blocks rather than wide tables. All headings, text, and code target US Letter portrait PDF output.

1. How to study this guide

Design principle = a rule for making choices, such as keeping responsibilities clear. **Design pattern** = a reusable way to organize classes or objects for a recurring problem. **Architecture pattern** = a way to organize larger components and their communication. An implementation may use several patterns, but every pattern should solve a named problem.

In the upcoming technical round, prepare to explain a design choice while coding or discussing a project. For the later system-design round, practice drawing boundaries, data flow, failure paths, and tradeoffs. Do not spend this week memorizing a pattern catalog. Learn the problem, the mechanism, the cost, and when to avoid the pattern.

Fit into the existing 20-hour plan

During Java fundamentals — 35 minutes. Study SOLID, composition, immutability, Strategy, Factory, Builder. Checkpoint: Refactor one conditional-heavy example aloud.

During Spring and API study — 35 minutes. Study layering, Adapter, Repository, dependency inversion, transactions. Checkpoint: Trace a request and name each responsibility.

During project-story practice — 30 minutes. Connect one real decision to its tradeoff and test. Checkpoint: Give a two-minute design story with personal ownership.

During production review — 35 minutes. Study outbox, retry, circuit breaker, bulkhead, idempotency. Checkpoint: Explain timeout and duplicate-event behavior.

During a mock follow-up — 15 minutes. Practice selecting and critiquing patterns. Checkpoint: Answer without buzzwords.

That is **2 hours 30 minutes** inside your 20-hour plan. For the later system-design round, revisit the architecture section more deeply. If the interviewer asks about a technology you used, give its actual implementation details before naming a pattern.

Use this answer structure: **"The problem was __. I chose __ because __. The cost was __. I handled failure by __. We verified it with __."** Speaking the tradeoff is what makes an answer sound senior.

2. Design principles you should know

1. Single responsibility principle (SRP) [High]

Plain English: A component should have one coherent reason to change. This does not mean it must have only one method. A payment class that validates requests, calculates fees, saves records, and sends email may mix unrelated changes.

Professional answer: "I group behavior around one responsibility and keep boundaries clear. For example, payment policy, persistence, and notification change for different reasons, so I separate them when that separation makes changes and tests safer."

When to use and cost: Split when changes repeatedly touch unrelated logic or tests become difficult. Too many tiny classes can make a simple flow hard to follow. Ask: "What business decision belongs here, and who owns it?"

Demonstration — a change request: A `PaymentService` both computes fees and sends approval emails. Finance asks for a new fee rule; Support asks for a different email template. Editing and retesting the same service for unrelated requests indicates two reasons to change. Move fee decisions to `FeePolicy` and notification delivery to `NotificationSender`; keep `SubmitPayment` responsible for coordinating the use case.

Why SRP applies: The split follows separate business owners and change cadence, not a rule that every method needs a class. **Evidence:** A fee-policy unit test no longer needs a mail client; a notification failure does not change the fee calculation. If neither behavior changes separately, the split might be premature.

Code anchor (illustrative fragment):

JAVA

```java
final class SubmitPayment {
    private final FeePolicy fees;
    private final PaymentStore store;

    void accept(Payment p) {
        Money fee = fees.calculate(p);
        store.save(p, fee);
    }
}
// Fee policy and persistence change separately.
```

Verify it: Change the fee formula: only the fee-policy tests should change. A mail client should never be required for this unit test.

2. Open/closed principle (OCP) [High]

Plain English: Add a new variation with limited changes to stable, existing code. If every new payment method forces edits to one large switch, the design may need an extension point.

Professional answer: "I use an extension point when variations are real and growing. A PaymentMethod interface can let a new method be added without repeatedly changing the orchestration. I still avoid abstractions for speculative future features."

Trap: A change in business policy may legitimately change existing code. OCP is a design direction, not a ban on editing a class.

Demonstration — growing payment channels: Initially `submit()` contains `if (type == CARD) ... else if (type == ACH) ...`. When a third supported channel arrives, model `PaymentHandler` with implementations for each channel and register supported handlers. The stable `SubmissionService` asks a resolver for the matching handler and invokes `submit()`.

Why OCP applies: New channel behavior is added mainly through a new implementation, rather than editing the core processing algorithm. **Evidence:** Existing channel tests pass unchanged and a new channel gets contract tests. The resolver or its configuration may still change; OCP does not promise literally zero edits.

Code anchor (illustrative fragment):

JAVA

```java
interface PaymentHandler {
    PaymentRail rail();

    void submit(Payment payment);
}

final class HandlerResolver {
    private final Map<PaymentRail, PaymentHandler> handlers;

    PaymentHandler forRail(PaymentRail rail) {
        return Objects.requireNonNull(handlers.get(rail));
    }
}
// Add another PaymentHandler implementation for a new rail.
```

Verify it: Register an additional rail and run the same handler contract tests without editing submission orchestration. Registration still needs a supported-rail entry.

3. Liskov substitution principle (LSP) [High]

Plain English: If B is used wherever A is expected, B should honor A's promises. A subclass that throws for a supported parent operation or silently changes its meaning breaks that promise.

Professional answer: "I design subtypes around behavioral contracts, not only matching method signatures. A replacement should preserve the expectations callers depend on, including error behavior and business invariants."

Example: A read-only account should not extend a mutable account contract with `withdraw()` that always throws. Separate capabilities or use composition. Test the same contract against each implementation.

Demonstration — broken substitution: A `MutableAccount.withdraw(amount)` contract promises to withdraw a valid amount or return a documented insufficient-funds result. `ReadOnlyAccount extends MutableAccount` instead throws `UnsupportedOperationException` for every withdrawal. A caller that was correct for `MutableAccount` now fails unexpectedly.

Better design: Give `AccountView` a `balance()` operation; make `WithdrawableAccount` a separate capability. A read-only account implements only `AccountView`. **Why LSP applies:** Each implementation can honor every operation it advertises. **Evidence:** Run shared contract tests against all `WithdrawableAccount` implementations, including boundary/error behavior.

Code anchor (illustrative fragment):

JAVA

```java
interface AccountReader {
    Balance balance(AccountId id);
}

interface AccountWriter {
    Receipt withdraw(AccountId id, Money amount);
}

final class ReadOnlyAccount implements AccountReader {
    public Balance balance(AccountId id) { /* ... */ }
}
// Read-only accounts never promise withdraw().
```

Verify it: Run the same read-contract tests for every AccountReader. Do not fake support for withdrawals by throwing UnsupportedOperationException.

4. Interface segregation principle (ISP) [Medium]

Plain English: A caller should depend on only the operations it needs. One giant financial-platform interface makes unrelated clients know too much.

Professional answer: "I prefer focused interfaces based on consumers. A reconciliation reader should not need payment-submission or user-administration methods. This makes dependencies and permissions easier to reason about."

Cost: Too many one-method interfaces can create noise. Split where consumers genuinely need different contracts.

Demonstration — unnecessary authority: A report job only needs `listSettledPayments()`, but is given a `FinancialPlatform` interface containing `submitPayment()`, `deleteUser()`, and `updateLimit()`. Even if unused, those operations enlarge the apparent dependency and may complicate permissions and mocks.

Better design: Inject `SettlementReader` into the report job; give payment submission a separate `PaymentSubmitter` interface. **Why ISP applies:** Each consumer sees its own legitimate operations. **Evidence:** The reporting test needs only a read fake and the report job cannot call a write operation through its dependency.

Code anchor (illustrative fragment):

JAVA

```
interface ReconciliationReader {
    List<Mismatch> findMismatches(BatchId batch);
}

interface PaymentSubmitter {
    Receipt submit(Payment payment);
}

final class ReportService {
    ReportService(ReconciliationReader reader) { /* ... */ }
}
// Reporting cannot accidentally submit payments.
```

Verify it: Compile the reporting module against only ReconciliationReader and confirm it cannot call payment submission methods.

5. Dependency inversion principle (DIP) [High]

Plain English: Business logic should depend on an appropriate contract rather than on a specific database, HTTP client, or vendor SDK. An adapter connects the contract to the outside system.

Professional answer: "I keep core business policy independent of infrastructure details. For example, a reconciliation service can depend on a `TransactionStore` contract while an adapter implements it with DB2 or another store. This allows focused tests and controlled replacement."

Clarify: Dependency injection supplies an implementation. Dependency inversion is the design of which component owns and depends on the interface. Constructor injection does not automatically make a design good.

Demonstration — a vendor replacement: Bad dependency: `ReconcileUseCase` imports `Db2JdbcClient` and contains DB2 SQL. Better dependency: it uses `TransactionStore.findUnmatched(batchId)`, a use-case-oriented contract; `Db2TransactionAdapter` owns SQL and row mapping. A test uses `InMemoryTransactionStore`.

Why DIP applies: The business-facing contract is controlled by the application, and the DB adapter depends on it—not the other way around. **Evidence:** The matching rule can be tested with no database; a storage migration changes the adapter but not reconciliation policy. Merely injecting `Db2JdbcClient` does **not** invert the dependency.

Code anchor (illustrative fragment):

JAVA

```
interface PaymentStore {
    Optional<Payment> find(PaymentId id);
}

final class PaymentInquiry {
    private final PaymentStore store;

    PaymentInquiry(PaymentStore store) {
        this.store = store;
```

```
    }
}

final class JdbcPaymentStore implements PaymentStore {
    public Optional<Payment> find(PaymentId id) { /* SQL */ }
}
```

Verify it: Replace JdbcPaymentStore with a fake in a use-case test. The domain/application code does not import JDBC.

6. Cohesion and coupling [High]

Plain English: Cohesion means related work stays together. Coupling means components rely on each other's details. Aim for cohesive components with narrow, understandable connections.

Professional answer: "I evaluate a boundary by asking whether its logic belongs together and how many implementation details leak across it. Clear contracts and data ownership reduce the number of changes needed for a new feature."

Example: If changing a database column requires edits in five unrelated services, a storage detail has leaked through boundaries. Low coupling does not mean no collaboration: services still need contracts.

Demonstration — schema leakage: The API response, fee algorithm, and audit logger all read `PAYMENT_FEE_RULE_CODE` directly from a database row. Renaming the column forces edits across unrelated modules. Introduce a repository mapping from the row to a domain concept such as `FeeRule`, then let other components consume the stable concept.

Why it fits: Mapping belongs with persistence (cohesion); the column name no longer leaks into pricing, transport, or audit (lower coupling). **Evidence:** A storage-column rename requires changing the repository mapping and its integration test, not the fee unit test or API contract.

Code anchor (illustrative fragment):

JAVA

```
// Persistence maps database vocabulary to domain vocabulary.
final class PaymentRowMapper {
    Payment toDomain(PaymentRow row) {
        return new Payment(row.id(),
                FeeRule.parse(row.feeRuleCode()));
    }
}
// FeeCalculator accepts Payment, not PaymentRow.
```

Verify it: Rename fee_rule_code in the schema and verify that the API and fee calculator stay unchanged.

7. Composition over inheritance [High]

Plain English: Composition builds behavior using collaborating objects. Inheritance models a true is-a relationship. Composition often makes policy selection easier without a deep class tree.

Professional answer: "I use composition for variable behavior, such as fee calculation or provider integration. I reserve inheritance for a stable substitutable relationship. This keeps each policy independently testable."

Example: `PaymentProcessor` uses `FeePolicy`. It does not need subclasses for every market, channel, and currency combination. Those axes can combine without multiplying subclasses.

Demonstration — combinatorial variation: An inheritance plan creates `DomesticCardProcessor`, `InternationalCardProcessor`, `DomesticAchProcessor`, and `InternationalAchProcessor`. Adding an approval-policy variation multiplies subclasses again. Prefer `PaymentProcessor(FeePolicy, PaymentRail, ApprovalPolicy)` and select each collaborator explicitly.

Why it fits: Region, rail, and authorization are independent axes. Composition avoids pretending every combination is a distinct subtype. **Evidence:** Add a new approval policy and test it against existing rails without duplicating their code. Avoid composition when it produces arbitrary combinations that the business actually forbids; validate compatibility at assembly time.

Code anchor (illustrative fragment):

JAVA

```java
final class PaymentProcessor {
    private final FeePolicy fees;
    private final ApprovalPolicy approvals;
    private final PaymentRailGateway rail;

    PaymentProcessor(FeePolicy fees,
                     ApprovalPolicy approvals, PaymentRailGateway rail) {
        this.fees = fees;
        this.approvals = approvals;
        this.rail = rail;
    }
}
```

Verify it: Change the approval policy without making new subclasses for every region and rail. Reject unsupported collaborator combinations at assembly time.

8. DRY, KISS, and YAGNI [High]

Plain English: DRY avoids repeating the same knowledge in many places. KISS favors the simplest design that meets the requirement. YAGNI says do not build a feature solely because it may be useful someday.

Professional answer: "I remove duplicated business rules, but I do not extract unrelated code just because two blocks look similar. I start with the simplest design that meets current behavior and add extension points when variation becomes real."

Trap: Two similar-looking transformations may follow different business policies. Forcing them into one shared helper can create many flags and make changes riskier. Duplicate syntax and duplicate knowledge are not always the same thing.

Demonstration — three different mistakes:

- **DRY:** Two services each hard-code the same legally required transaction rounding rule. Put the single policy definition in an owned domain component and test it once with boundary cases.
- **KISS:** A fixed two-case mapping is clearer as a small switch than a plugin framework with discovery, reflection, and configuration.
- **YAGNI:** Do not build a multi-region payment router before the business supports a second region or has a credible migration requirement.

Why these differ: DRY concerns duplicated knowledge, KISS concerns unnecessary complexity, and YAGNI concerns speculative requirements. **Evidence:** A real policy change needs one source update, the code remains readable, and no extension machinery exists without a current use case.

Code anchor (illustrative fragment):

JAVA

```java
// DRY: one owner for a real shared business rule.
Money roundFee(Money input) { /* agreed rule */ }

// KISS: for two stable options, a switch is enough.
boolean needsReview(RiskBand band) {
    return switch (band) {
        case HIGH -> true;
        case LOW -> false;
    };
}
// YAGNI: no plugin registry without a real second plugin.
```

Verify it: Test one rounding policy with boundary values. Avoid building extension machinery to replace a readable two-case switch.

9. Encapsulation and invariants [High]

Plain English: Encapsulation hides implementation details and protects rules that must always hold. An invariant is such a rule: an approved payment cannot have a negative amount, or an account change must be authorized.

Professional answer: "I keep state changes behind operations that enforce the business rules. I do not expose mutable internals and hope every caller validates them correctly. I also use database constraints for rules that must hold across concurrent application instances."

Example: A `Payment` may expose `approve()` with checks rather than a public `setStatus()`. Client-side validation and DTO validation do not replace domain or database protection.

Demonstration — illegal state: A publicly writable `payment.status` permits a caller to set `SETTLED` on a draft without authorization. Replace arbitrary setters with `authorize(actor)`, `markSettled(providerReference)`, and guarded transitions. Persist conditional updates to handle competing processors.

Why it fits: An invariant such as "settlement requires authorization and a provider reference" is enforced at the state-changing boundary. **Evidence:** Tests reject invalid transitions; a concurrent update test confirms only the expected prior version can advance. Object-level checks alone do not prevent database races; use version checks, locking, or constraints as appropriate.

Code anchor (illustrative fragment):

JAVA

```java
final class Payment {
    private Status status = Status.CREATED;

    void authorize() {
        if (status != Status.CREATED)
            throw new IllegalStateException("bad transition");
        status = Status.AUTHORIZED;
    }
    // No public setStatus(...).
}
```

Verify it: An attempt to authorize twice must fail. Add database concurrency protection when two processes can update the same row.

10. Design for testability and observability [High]

Plain English: A useful design makes correct behavior and failure behavior visible. Small contracts make tests easier; logs, metrics, and traces make production behavior explainable.

Professional answer: "I build seams around external dependencies and test business rules independently. I also make important state transitions and failure paths observable so an incident can be diagnosed from evidence rather than guesses."

Example: Test a retry rule with a fake provider, then integration-test real persistence and deduplication constraints. Track request ID, business ID, state, latency, and error category without logging secrets.

Demonstration — an intermittent timeout: Rather than performing a real provider call in a unit test, inject a `ClearingGateway` port whose fake returns success, a transient error, or an unknown outcome. Verify the correct business status for each case. In production, emit a trace with instruction ID, dependency name, latency, and a sanitized outcome category.

Why it fits: Tests prove the decision path, while telemetry helps explain the same path under live load. **Evidence:** A test reliably reproduces timeout behavior and an operator can correlate a pending instruction to its retries without exposing bank or identity secrets. Unit tests do not replace contract or integration tests.

Code anchor (illustrative fragment):

JAVA

```java
interface ClearingGateway {
    GatewayResult send(Payment p);
}
```

```
// In a unit test:
ClearingGateway timeout = p -> GatewayResult.UNKNOWN;
PaymentService service = new PaymentService(timeout);

assertEquals(Status.OUTCOME_UNKNOWN, service.submit(testPayment()));
```

Verify it: Inject a fake timeout and assert the business state. In production, log the instruction ID and sanitized error category, never account secrets.

11. Fail fast at boundaries, recover deliberately [Medium]

Plain English: Reject invalid input before doing expensive or irreversible work. For failures that may be temporary, recover only when the operation is safe to repeat.

Professional answer: "I validate preconditions early, return a clear error for invalid requests, and reserve retries for known transient failures. For a timed-out payment call, I first determine whether the operation completed before retrying."

Trap: "Fail fast" does not mean crash every service on a recoverable downstream problem. The boundary, recovery contract, and customer impact matter.

Demonstration — validation versus uncertainty: Reject a missing beneficiary or unsupported currency with a clear client error before storing an instruction. By contrast, if a remote provider times out after a valid submission, record `OUTCOME_UNKNOWN`; do not treat it as an invalid request or blindly re-send it.

Why it fits: Invalid input is deterministic; a remote timeout may conceal a successful side effect. **Evidence:** An invalid-input test proves no instruction/event was persisted, while a timeout test proves the system schedules a status query or reconciliation. A retry is allowed only if the target's deduplication contract makes repetition safe.

Code anchor (illustrative fragment):

JAVA

```
SubmitResult submit(Request request) {
    if (request.beneficiary() == null)
        throw new BadRequestException("beneficiary required");
    GatewayResult result = gateway.send(toPayment(request));
    if (result == GatewayResult.UNKNOWN)
        return SubmitResult.pendingReconciliation();
    return SubmitResult.from(result);
}
```

Verify it: The bad input path must make zero gateway calls. The unknown-outcome path must not blindly retry a potentially completed payment.

12. Separate policy from mechanism [Medium]

Plain English: Policy is the business decision; mechanism is how the system performs it. "Who may approve a payment?" is policy. "How do we read the role from a token?" is mechanism.

Professional answer: "I keep business decisions explicit and avoid burying them in transport or persistence code. A policy can then be reviewed by business owners and tested without making an HTTP call or using a real database."

Example: `ApprovalPolicy.canApprove(principal, payment)` should express the rule. A security adapter obtains the principal, and a repository loads the payment.

Demonstration — approval access: A controller inspects JWT claims and contains `if (role == MANAGER && amount < threshold) ...`. That mixes token parsing with a business approval rule. Instead, a security adapter resolves `Principal`, the application loads `Payment`, and `ApprovalPolicy.canApprove(principal, payment)` decides.

Why it fits: Business owners can review the approval policy independently of token format, and an identity-provider migration changes the adapter. **Evidence:** Table-driven tests vary role, tenant, limit, and payment state with no HTTP request. Authentication still belongs at the boundary; do not move token parsing into the domain.

Code anchor (illustrative fragment):

JAVA

```java
// Business rule: no JWT parsing here.
boolean canApprove(Principal actor, Payment p) {
    return actor.tenant().equals(p.tenant())
            && actor.approvalLimit().compareTo(p.amount()) >= 0;
}
// HTTP/JWT adapter builds Principal; policy uses it.
```

Verify it: Vary role, tenant, limit, and payment status in table-driven tests with no HTTP request.

3. Object-level design patterns

Patterns are useful vocabulary, not proof of good design. In this section, say **problem → shape → example → cost**. When an interviewer asks "Which pattern would you use?", clarify whether behavior varies, a vendor must be isolated, work is asynchronous, or state changes are constrained.

Strategy [High]

Plain English: Put each interchangeable rule behind the same small contract. Select the rule based on context.

Professional answer: "I use Strategy when one behavior has several valid policies, such as fee calculation by product. The caller uses one contract and each policy is tested separately. I avoid it for a single simple branch."

Shape: `FeePolicy.calculate(payment)`. Implementations could be domestic and international fee policies. Be careful that variation may depend on more than country; make selection rules explicit. **Cost:** More classes and a policy-selection mechanism.

Demonstration: A service must calculate different fees for domestic and cross-border instructions. It asks `FeePolicy.calculate(instruction)`; `DomesticFeePolicy` and `CrossBorderFeePolicy` implement that contract, and a `FeePolicySelector` chooses according to a documented rule.

Why Strategy, not State: The rule changes with instruction context; the payment does not transition from one policy object to another through its lifecycle. **Proof:** Run the same fee contract tests on both policies, plus selector tests at region/currency boundaries. **Avoid it:** A stable one-line conditional may be clearer.

Code anchor (illustrative fragment):

JAVA

```java
interface FeePolicy {
    Money fee(Payment p);
}

final class DomesticFee implements FeePolicy {
    public Money fee(Payment p) {
        return Money.usd("1.50");
    }
}

final class CrossBorderFee implements FeePolicy {
    public Money fee(Payment p) {
        return p.amount().times("0.01");
    }
}
// A resolver chooses the rule; processor calls policy.fee(p).
```

Verify it: Change one fee rule without changing the processor. Test policy selection separately from fee computation.

Factory Method or simple factory [High]

Plain English: Put object creation and selection in one place. A simple factory can choose a `PaymentHandler` based on payment type. Factory Method specifically lets subclasses decide which concrete object to create.

Professional answer: "I separate complex creation from the business flow. If selection rules are centralized, a factory can return the correct handler while the processing code works with the handler contract. For a trivial object, direct construction is clearer."

Trap: Do not call every constructor wrapper a Factory Method. An interviewer may ask you to distinguish the formal pattern from a simple factory.

Demonstration: A `HandlerFactory.forRail(rail)` returns an `AchHandler` or `CardHandler`. The submission use case asks for a handler without knowing its constructor dependencies. This is a **simple factory**, because a selector method—not overriding subclasses—chooses the result.

Why a factory: Handler creation has real selection rules or nontrivial configuration. **Proof:** Unsupported rails receive an explicit rejection, and mapping tests show the correct handler for every supported rail. **Avoid it:** Construct `new FeeCalculator()` directly if no selection or construction complexity exists.

Code anchor (illustrative fragment):

JAVA

```java
final class PaymentHandlerFactory { // Simple factory.
    PaymentHandler create(PaymentRail rail) {
        return switch (rail) {
            case ACH -> new AchHandler();
            case CARD -> new CardHandler();
        };
    }
}
// Factory Method specifically delegates creation to subclasses.
```

Verify it: Verify the returned concrete handler for each supported rail; avoid a factory when one constructor is sufficient.

Builder [High]

Plain English: Construct a complex object step by step while making required data and validation clear.

Professional answer: "I use Builder when a request or configuration has many optional fields and construction should be readable. I validate required fields at build time and favor an immutable result."

Example: A payment instruction has mandatory tenant, amount, currency, and beneficiary, plus optional remittance details. **Cost:** Extra code; records or constructors may be enough for a small object.

Demonstration: A `PaymentInstruction` needs tenant, beneficiary, currency, and amount, with optional remittance text and delivery preference. A builder makes call sites readable: `builder.tenant(t).beneficiary(b).amount(a).currency(c).build()`. `build()` rejects missing mandatory fields and returns an immutable result.

Why Builder, not Factory: The question is how to assemble and validate one complex value, not which implementation to choose. **Proof:** Tests check missing required fields, invalid combinations, and that the resulting value cannot be changed through shared mutable references. A short Java record or constructor may be better when there are only a few fields.

Code anchor (illustrative fragment):

JAVA

```java
PaymentInstruction instruction = PaymentInstruction.builder()
        .tenantId(tenantId)
        .amount(amount)
        .currency("USD")
        .beneficiary(beneficiary)
```

```
        .reference("invoice-42") // optional
        .build(); // validates required fields
```

Verify it: Omit each mandatory field in turn: build () must reject the incomplete value. Keep the built object immutable.

Adapter [High]

Plain English: Translate an external interface into the interface your application wants.

Professional answer: "I use Adapter to keep provider-specific APIs and data shapes outside business logic. Our core service calls a stable internal contract; the adapter handles provider request mapping, errors, and transport details."

Example: An old file-based payment interface and a new real-time service can implement the same internal `PaymentSubmissionPort` during migration. **Cost:** Translation logic and another layer; do not hide differences that change business semantics.

Demonstration: Provider A accepts `{customerRef, totalMinorUnits}` and returns status `QUEUED`; Provider B expects an XML payment file and returns a receipt code. Both are mapped to the internal `ClearingGateway.submit(Instruction)` contract and a carefully defined `SubmissionReceipt`.

Why Adapter: Translation isolates external syntax and transport from the core. **Proof:** Provider-specific contract tests cover units, currency, provider errors, and status mapping; core tests use a fake `ClearingGateway`. **Important:** If one provider's `QUEUED` means only receipt rather than confirmed execution, preserve that semantic difference rather than mapping it to `SETTLED`.

Code anchor (illustrative fragment):

JAVA

```java
interface ClearingGateway {
    GatewayResult submit(Payment p);
}

final class LegacyFileAdapter implements ClearingGateway {
    private final LegacyFileClient client;

    public GatewayResult submit(Payment p) {
        return mapResponse(client.writeLine(toLegacyLine(p)));
    }
}
```

Verify it: Test field and error mapping. The use case must not depend on the legacy file format.

Facade [High]

Plain English: Give callers one simple entry point to a complicated subsystem. The facade coordinates existing capabilities.

Professional answer: "I use a facade when clients should not orchestrate several low-level components themselves. For example, a user-onboarding facade can coordinate identity creation, role assignment, and audit recording behind one application operation."

Cost: A facade can become a very large service if it absorbs all business rules. Keep each collaborator's responsibility visible. A facade simplifies access; an adapter changes an interface.

Demonstration: New-user onboarding requires `IdentityCreator`, `RoleAssigner`, and `AuditRecorder`. Instead of requiring a controller to invoke them in the right order, expose `OnboardingFacade.createUser(request)` and let it coordinate their interfaces.

Why Facade: Callers need one business-level operation; no external interface needs translation. **Proof:** An application test verifies the call order and failure handling; each collaborator has its own tests. **Avoid it:** Do not hide every permission rule and persistence detail in a giant facade. In contrast, an Adapter primarily translates one interface into another.

Code anchor (illustrative fragment):

JAVA

```java
final class OnboardUserFacade {
    UserId onboard(NewUser input) {
        UserId id = identity.create(input);
        roles.assignDefaultRole(id);
        audit.recordUserCreated(id);
        return id;
    }
}
// One use-case entry point; collaborators keep their rules.
```

Verify it: Confirm that onboarding calls the collaborators in the required order and does not duplicate their domain rules.

Decorator [Medium]

Plain English: Wrap a component with another component that has the same contract and adds behavior.

Professional answer: "Decorator can add cross-cutting behavior such as metrics or caching around a repository or client without changing its core implementation. I use it when that behavior is composable and the wrapper's effect remains clear."

Example: `TimedPaymentGateway` measures a gateway call and delegates to `PaymentGateway`. **Cost:** Several wrappers can obscure call order and failure behavior. A cache wrapper needs a freshness policy.

Demonstration: `TimedGateway` implements `ClearingGateway`, contains another `ClearingGateway`, records elapsed time and outcome, then delegates `submit()`. A `TracingGateway` could wrap the same contract separately.

Why Decorator: The caller still sees the original contract, while behavior is composed around it. **Proof:** The delegate is called once, the return value/error behavior is preserved, and metrics are recorded on success and failure. **Cost:** Wrapper order matters. For example, timing outside a retry wrapper measures total retry duration, while timing inside measures each attempt.

Code anchor (illustrative fragment):

JAVA

```java
final class TimedGateway implements ClearingGateway {
    private final ClearingGateway delegate;

    public GatewayResult submit(Payment p) {
        long start = clock.nanoTime();
        try {
            return delegate.submit(p);
        } finally {
            metrics.record(clock.nanoTime() - start);
        }
    }
}
```

Verify it: Measure both successful and failed calls. Verify that decoration does not change the gateway result.

Proxy [Medium]

Plain English: A proxy stands in front of another object and controls access to it. It may add transactions, security, lazy loading, or remote access.

Professional answer: "I expect proxy behavior at framework boundaries. In Spring, transaction advice commonly runs through a proxy, so a method calling another method on the same instance may bypass that advice. I test transaction behavior at the actual call boundary."

Difference from Decorator: Both wrap, but a proxy often controls access or lifecycle, whereas a decorator intentionally adds composable behavior. The structures can look similar; discuss intent and effects. See Spring transaction implementation.

Demonstration: `@Transactional` is placed on `PaymentWriter.commit()`, but another method on the same instance calls `this.commit()`. With normal Spring proxy-based transaction management, that internal call bypasses transactional interception. Call the transactional method through a separately injected Spring-managed bean (or design a correctly advised external use-case boundary).

Why Proxy: Spring intercepts calls that pass through the proxy; it is a runtime access boundary rather than a fee-rule extension. **Proof:** An integration test forces a database error and checks whether the expected writes roll back. Verify configuration and transaction manager behavior; annotations alone are not proof.

Code anchor (illustrative fragment):

JAVA

```
@Service
class PaymentService {
    void accept() {
        saveInTransaction();
    } // Self-call bypasses proxy.

    @Transactional
    public void saveInTransaction() { /* DB writes */ }
}
// Prefer an external call through a proxied bean, or a
// correctly placed public transactional use-case boundary.
```

Verify it: Integration-test rollback through a real Spring proxy, not merely a direct method call. Official Spring docs confirm proxy self-invocation behavior.

Observer and publish/subscribe [Medium]

Plain English: One component announces an event and interested components react. In-process Observer means listeners in the same process. A message broker makes communication cross-process, with different durability and delivery concerns.

Professional answer: "I publish an event when other components need to react to a completed state change without being called synchronously. I distinguish a local callback from a durable event and plan for duplicates, ordering, and failed consumers."

Example: User-created event triggers audit or notifications. Do not send a durable event before its source transaction commits unless the design handles rollback.

Demonstration: After `UserCreated` is durably committed, an audit consumer records the change and a notification consumer sends a welcome message. Each subscribes to the fact without the user-creation use case having to call them synchronously.

Why pub/sub: Independent reactions can lag without blocking creation. **Proof:** Duplicate `UserCreated` delivery does not create two audit effects or duplicate customer-facing notifications (by consumer-side idempotency). **Boundary:** A local Java listener has a different durability guarantee than broker delivery; simply calling `publish()` before a DB commit can produce a false event if the transaction rolls back.

Code anchor (illustrative fragment):

JAVA

```
record PaymentAccepted(PaymentId id, UUID eventId) {
}
// After source state commits, publish a durable event.
subscriber.

on(PaymentAccepted event) {
    if (dedup.insertIfAbsent(event.eventId()))
        projection.update(event.id());
}
// Dedup and side effect must be atomic in real storage.
```

Verify it: Deliver the same event twice: the projection must record one effect. This code is broker-neutral pseudocode for the callback body.

Command [Medium]

Plain English: Represent an operation as a value. This can separate receiving a request from executing, recording, retrying, or scheduling it.

Professional answer: "A command makes the requested action explicit, such as `ApprovePayment`. A handler validates and executes it. This can help with workflow, auditing, or async dispatch, but I do not add command objects to simple code without a need."

Trap: A command is an instruction or intent; an event states something that already happened. `ApprovePayment` and `PaymentApproved` are not interchangeable.

Demonstration: `ApprovePayment(paymentId, approverId, requestId)` describes a requested action. An `ApprovePaymentHandler` checks permissions and state, writes an approved transition, and may later emit `PaymentApproved(paymentId, approvedAt)`.

Why Command: The request can be validated, traced, queued, or retried as a distinct unit of work. **Proof:** Handling the same request identifier twice does not create duplicate approval effects; unauthorized commands are rejected. **Key distinction:** `ApprovePayment` can fail; `PaymentApproved` claims that approval already happened.

Code anchor (illustrative fragment):

JAVA

```java
record ApprovePayment(PaymentId id, UserId actor) {
}

final class ApprovalHandler {
    void handle(ApprovePayment command) {
        Payment p = store.load(command.id());
        policy.requireAllowed(command.actor(), p);
        p.approve();
        store.save(p);
    }
}
```

Verify it: Run unauthorized and duplicate command tests. A command asks for work; it does not claim that work already happened.

State [High]

Plain English: Behavior changes depending on an object's state. Representing valid transitions prevents scattered status checks.

Professional answer: "I model payment status as explicit transitions and reject invalid changes. For a complex workflow, a State pattern or transition table can make the rules reviewable. For a small fixed workflow, an enum with a validated transition method may be simpler."

Example: `CREATED -> AUTHORIZED -> SETTLED`, with `FAILED` paths. Define whether reversal after settlement is a new compensating operation rather than simply moving status backward. **Cost:** More structure, especially if every status becomes a class.

Demonstration: An instruction has `CREATED`, `AUTHORIZED`, `SUBMITTED`, `SETTLED`, and `REJECTED` states. `settle()` is legal only after a confirmed provider outcome; `submit()` from `SETTLED` is invalid. Encode the transition table in a domain method or, if behavior is complex, separate State objects.

Why State, not Strategy: Behavior is constrained by the object's lifecycle, not a freely chosen policy. **Proof:** Test every permitted and forbidden state/action pair, including competing updates. **Avoid it:** Do not generate one class per enum value when a transition table expresses the rules clearly.

Code anchor (illustrative fragment):

JAVA

```java
enum Status {CREATED, AUTHORIZED, SETTLED}

boolean allowed(Status from, Status to) {
    return switch (from) {
        case CREATED -> to == Status.AUTHORIZED;
        case AUTHORIZED -> to == Status.SETTLED;
        case SETTLED -> false;
    };
}
```

Verify it: Test every allowed and forbidden edge. A fixed enum can be better than one State subclass per status.

Template Method [Medium]

Plain English: A base class fixes the overall algorithm and lets subclasses fill certain steps.

Professional answer: "Template Method fits a stable process with a few controlled variations. For example, each import may share validate, transform, persist, and report steps. I prefer composition if the variations combine in many ways or inheritance makes the hierarchy brittle."

Trap: A subclass must preserve the base process's contract. Hidden calls from the base class to overridable methods can complicate debugging.

Demonstration: `BaseStatementImporter.importFile()` fixes `validate -> parse -> persist -> summarize`. `CsvImporter` and `FixedWidthImporter` override only parsing; neither can bypass the shared validation in the base algorithm.

Why Template Method: The process order is stable, with a few controlled algorithm steps. **Proof:** Tests assert that invalid files never reach `persist()`, regardless of concrete importer. **Limit:** If validation, transformation, and persistence vary independently, composed strategies are easier to test and combine than a deep importer hierarchy.

Code anchor (illustrative fragment):

JAVA

```java
abstract class ImportJob {
    public final void run(File file) {
        validate(file);
        persist(transform(file));
        report();
    }

    protected abstract Rows transform(File file);
}
// Shared sequence is fixed; subclasses vary one step.
```

Verify it: A subclass may change transformation but not reorder validation, persistence, and reporting.

Chain of Responsibility [Medium]

Plain English: Pass a request through a sequence of handlers; each handler may process, reject, or pass it on.

Professional answer: "I use a chain when independent checks need an ordered pipeline, such as authentication, input limits, and audit enrichment. I document the order and whether a handler can stop processing."

Cost: Debugging becomes hard if order is implicit or failures disappear in the chain. Spring filters or interceptors are familiar examples, though their exact semantics differ.

Demonstration: An API request passes through `Authenticate -> TenantAuthorize -> ValidateSize -> RateLimit -> Controller`. Each step can allow the request to continue or reject it with an explicit error.

Why Chain: Multiple focused handlers must run in a known order. **Proof:** A test with an invalid tenant shows the request stops before the controller; logs identify which handler rejected it. **Risk:** Rate-limiting before authentication may require a different key and have different security properties; filter order is a design decision, not an accident.

Code anchor (illustrative fragment):

JAVA

```java
interface Check {
    void verify(Request request);
}

final class ValidationPipeline {
    private final List<Check> checks;

    void run(Request request) {
        for (Check check : checks) check.verify(request);
    }
}
// Throwing from a check stops later checks.
```

Verify it: Test order and early exit. Security checks must not be accidentally skipped because of chain configuration.

Repository and Unit of Work [High]

Plain English: A Repository gives domain-oriented access to stored objects. A Unit of Work tracks related changes that should commit as one unit. In Spring/JPA, a persistence context and transaction often supply much of the Unit of Work behavior.

Professional answer: "I keep persistence queries behind meaningful repository operations and place the transaction around the business operation. I do not assume a repository method makes a multi-step workflow atomic; that depends on the transaction boundary and database guarantees."

Cost: A generic repository with dozens of vague methods can hide inefficient queries. A repository is not automatically an abstraction over every storage engine.

Demonstration: `ApprovePaymentUseCase` loads an instruction through `PaymentRepository`, changes its domain state, and appends an audit/outbox record inside **one local transaction**. A call to `save()` alone does not make the entire multi-step operation atomic.

Why Repository + Unit of Work: Repository hides storage-specific queries; a transaction coordinates the related state changes. **Proof:** Force an outbox insert failure and verify the approval update also rolls back. Add a concurrent-approval test with version checks or suitable locking. An HTTP call to an external provider is not automatically part of that transaction.

Code anchor (illustrative fragment):

JAVA

```java
@Service
final class SettlementService {
    @Transactional
    public void settle(PaymentId id) {
        Payment p = repository.findRequired(id);
        p.markSettled();
        repository.save(p);
        outbox.save(PaymentSettled.of(p));
    }
}
// The local DB transaction covers both writes.
```

Verify it: Force an outbox insert failure: the payment update must roll back. A broker publish is not inside this local transaction.

Singleton: recognize it, use sparingly [Medium]

Plain English: A Singleton controls creation so there is one instance within a defined scope. In a distributed service, each process still has its own instance.

Professional answer: "I usually let the application container manage shared, stateless services. I do not use a global singleton for request state or payment deduplication because it cannot coordinate separate instances or survive restarts."

Trap: A Spring singleton bean is scoped to an application context. Shared mutable fields still need concurrency protection.

Demonstration: A stateless `FeeCatalog` can be a shared Spring bean. A mutable `processedIds = new HashSet<>()` stored in a singleton is **not** safe payment deduplication: concurrent requests race, each service instance has a separate set, and restarts erase it.

Why it matters: Instance scope is not a distributed consistency mechanism. **Proof:** Run the duplicate-request test across two app instances; expect a database uniqueness rule or other durable, coordinated store to decide the winner. Avoid storing per-request tenant or customer context in singleton fields.

Code anchor (illustrative fragment):

JAVA

```java
@Service // Spring singleton by default: per app context.
final class FeeCalculator {
    Money calculate(Payment payment) { /* stateless */ }
}
// BAD: a mutable field such as currentTenant
// would mix data from concurrent requests.
```

Verify it: Send concurrent requests for two tenants; a shared service must not retain tenant-specific mutable state.

4. Architecture patterns and when to choose them

1. Layered architecture [High]

Plain English: Group work by role, such as API, application service, business model, and persistence. A request moves through clear responsibilities.

Professional answer: "Layering helps keep transport, business rules, and data access understandable. In a Spring service, the controller handles HTTP, the application service coordinates a use case, and repositories persist state. I watch for business rules leaking into controllers or repositories."

Use when: A service has a straightforward workflow. **Cost:** Too many pass-through layers; an anemic domain model if every rule is pushed into one large service class.

Demonstration: `PaymentController` parses HTTP and returns an API response; `SubmitPaymentService` authenticates business intent and coordinates the use case; `Payment` enforces valid state transitions; `PaymentRepository` reads/writes data.

Why layering: It prevents controllers from becoming business engines and repositories from deciding payment policy. **Proof:** A fee-rule change can be tested without an HTTP server; a request-shape change does not alter domain transitions. **Tradeoff:** Avoid controller → service → service → repository pass-through chains that do not add a meaningful boundary.

Code anchor (illustrative fragment):

JAVA

```java
@RestController
class PaymentController {
    private final SubmitPayment useCase;

    @PostMapping("/payments")
    Receipt submit(@RequestBody PaymentRequest req) {
        return useCase.submit(req.toCommand());
    }
}
// Controller: HTTP; use case: workflow; repository: storage.
```

Verify it: A policy change should not require editing HTTP mapping or SQL. Avoid layers that do nothing beyond forwarding.

2. Hexagonal architecture / ports and adapters [High]

Plain English: Put core business logic in the center. Define ports for what it needs and adapters for HTTP, databases, queues, or vendor systems. The core should not need to know the vendor's API.

Professional answer: "I use ports and adapters when external integrations or storage may change, or when business rules need isolated tests. The application owns a business-oriented port, and an adapter translates to the technology. I do not add a separate interface and adapter for every trivial class."

Example: `SubmitPaymentUseCase` depends on `PaymentStore` and `ClearingGateway`. REST and batch entry points call the same use case. DB2 and provider adapters implement the ports. **Cost:** More boundary code and mapping. See AWS hexagonal architecture guidance.

Demonstration: Two entry adapters, REST and batch-file import, both invoke `SubmitPaymentUseCase`. The use case depends on `PaymentStore` and `ClearingGateway` ports. `JdbcPaymentAdapter` and `ProviderHttpAdapter` implement those ports; neither transport leaks into fee and authorization rules.

Why hexagonal: The application owns stable capabilities while technologies sit outside. **Proof:** The same contract tests run against a fake store and the actual JDBC adapter; changing REST to batch input leaves the core use case unchanged. **Tradeoff:** Boundaries and mapping require code; a small CRUD endpoint may not need all those ports.

Code anchor (illustrative fragment):

JAVA

```java
// Owned by the application core:
interface ClearingPort {
```

```
    GatewayResult clear(Payment p);
}

final class SubmitPayment {
    private final ClearingPort clearing;

    SubmitPayment(ClearingPort clearing) {
        this.clearing = clearing;
    }
}
// ProviderSdkAdapter implements ClearingPort at the edge.
```

Verify it: Instantiate the use case in a test with an in-memory port and no provider SDK. The dependency points inward.

3. Modular monolith [High]

Plain English: One deployable application can still have strong internal modules, such as identity, payment, and reconciliation. Each module owns its rules and communicates through clear interfaces.

Professional answer: "I consider a modular monolith when teams need clean boundaries but separate deployments and distributed failure handling are not justified. It is a useful starting point because local transactions and debugging are simpler. I keep module ownership explicit so future extraction remains possible."

Cost: Shared deployment and runtime; boundaries can erode if every module reads another's tables. A module is not isolated merely because it has a package name.

Demonstration: One Spring deployment contains `identity`, `payments`, and `reconciliation` modules. Reconciliation calls an exported `payments` query interface instead of reading payment tables directly; module tests reject forbidden imports.

Why modular monolith: Teams gain explicit ownership without network latency and distributed transactions for every interaction. **Proof:** A payment schema refactor does not require edits in the reconciliation module; module dependency rules are enforced in CI. **Tradeoff:** The application still deploys and fails as one process. Extract a service only when the independent deployment or scaling need is proven.

Code anchor (illustrative fragment):

JAVA

```
// identity module exposes an application-level interface:
interface IdentityLookup {
    Principal findAuthorized(UserId actor);
}

// payments module depends on IdentityLookup,
// not identity's private tables or persistence classes.
final class SubmitPayment {
    SubmitPayment(IdentityLookup identities) { /* ... */ }
}
```

Verify it: Add a module-boundary test: payments must not import identity.persistence.* or query its tables directly.

4. Microservices with owned data [High]

Plain English: Independent services own capabilities and, ideally, control their own data. They communicate through APIs or events. This helps independent change but creates network, consistency, testing, and operations costs.

Professional answer: "I choose service boundaries around business capabilities and team ownership, not around small classes. I account for API contracts, data ownership, observability, deployment, and partial failure. I do not split a cohesive workflow simply because microservices are popular."

Caution: Database-per-service means one service controls access to its data; it does not require a separate database server for every service. Cross-service reports need API composition, materialized views, or events. See database-per-service pattern.

Demonstration: The Payment service owns instructions; Reconciliation owns matching runs; Identity owns user access data. Reconciliation consumes `PaymentSettled` events or a supported Payment API rather than querying Payment tables directly.

Why microservices: Separate teams may need independent releases, capacity, and fault isolation. **Proof:** The reconciliation deploy leaves Payment unchanged; a reconciliation outage does not corrupt accepted payments, and lag is monitored. **Tradeoff:** Cross-service reports require projections/APIs, events may be delayed, and failure recovery is harder. A split that causes constant synchronous calls may be worse than a modular monolith.

Code anchor (illustrative fragment):

EXAMPLE

```
Payment Service --HTTP/event--> Identity Service
      |                              |
Payment-owned tables         Identity-owned tables

// No cross-service SQL join or shared-table writes.
// Payment may cache a permitted identity projection.
```

Verify it: Stop Identity Service: Payment must have a defined timeout or degraded response. Teams must own API versioning and data boundaries.

5. Synchronous request/response [High]

Plain English: A caller sends a request and waits for an answer. It is simple when the answer is needed now. The caller's response time and availability depend on the downstream path.

Professional answer: "I use a synchronous call when the caller needs an immediate decision. I set a deadline, bound concurrency, and define what a timeout means. I avoid a long chain of dependent calls when it makes latency and failure hard to control."

Example: Validate a payment request and return whether it was accepted for processing. Do not promise final settlement if that occurs later.

Demonstration: `POST /payment-instructions` synchronously authenticates, validates, and commits an instruction record. It returns an ID and `ACCEPTED` once local acceptance is durable; settlement occurs asynchronously.

Why synchronous for acceptance: The caller must know whether its request was durably recorded before leaving the screen. **Proof:** A successful response always maps to a retrievable instruction, and a commit failure never produces a successful acknowledgment. **Limit:** Provider settlement should not be implied by the acceptance HTTP status. Long dependency chains can consume the caller's latency budget.

Code anchor (illustrative fragment):

JAVA

```java
HttpRequest request = HttpRequest.newBuilder(uri)
        .timeout(Duration.ofSeconds(2))
        .POST(BodyPublishers.ofString(payload))
        .build();
HttpResponse<String> response =
        client.send(request, BodyHandlers.ofString());
// Timeout is a caller deadline, not proof of remote failure.
```

Verify it: Verify request deadlines and distinguish accepted-for-processing from fully settled. Client and transport settings may add other timeouts.

6. Event-driven and publish/subscribe architecture [High]

Plain English: A service emits a fact about a completed change; other services respond when they receive it. This reduces direct coupling but introduces delivery delay and duplicate or out-of-order events.

Professional answer: "I use events when downstream consumers can react independently and eventual consistency is acceptable. I define event ownership, schema evolution, idempotent consumers, ordering needs, and how failed events are replayed."

Example: `PaymentAccepted` can trigger audit and status projection. Do not put sensitive data in an event merely for convenience. A command tells one component to do something; an event tells interested components something happened.

Demonstration: After a payment settles, the Payment service emits `PaymentSettled` with event ID, instruction ID, schema version, and non-sensitive status metadata. A reconciliation projection and an audit consumer update independently.

Why events: Payment does not need to wait for the two reactions to finish. **Proof:** Replay the event and verify that idempotent consumers do not duplicate their results; a delayed event shows an explicit 'last updated' time on the projection. **Limit:** An event is not a synchronous query or a guarantee of immediate read consistency.

Code anchor (illustrative fragment):

JAVA

```java
record PaymentAccepted(PaymentId id, UUID eventId) {
}

void onPaymentAccepted(PaymentAccepted event) {
    if (!inbox.claimOnce(event.eventId())) return;
    auditProjection.apply(event.id());
}
// Claim and projection update need an atomic strategy.
```

Verify it: Replay an event and deliver it out of order. Decide whether ordering matters and how projection versioning handles it.

7. Transactional outbox [High]

Plain English: Saving business data and publishing an event are two writes. One can succeed while the other fails. An outbox saves the business change and event record in one local database transaction; a relay publishes the event afterward.

Professional answer: "I use an outbox when a committed state change must reliably produce an event. The local transaction writes both business and outbox rows. A relay may publish more than once, so consumers still need idempotency and monitoring."

Cost: Outbox storage, relay, lag, cleanup, ordering decisions. It does not make an external payment provider part of the local transaction. See AWS transactional outbox.

Demonstration — dual-write failure: If `savePayment()` commits but `broker.publish()` fails, downstream consumers never hear about it. Instead, within a **single database transaction**, insert the payment row and an outbox row containing `eventId`, type, and payload. A separate relay publishes pending records and marks or tracks delivery.

Why outbox: It closes the gap between committed local state and a missing event. **Proof:** Kill the process after DB commit and before publish: the relay still finds the event; publish twice and verify the consumer effect occurs once. **Limit:** The relay can redeliver, so it is **not** a blanket exactly-once guarantee. Monitor relay lag and retention.

Code anchor (illustrative fragment):

SQL

```sql
BEGIN;
INSERT INTO payment_instruction(id, status)
VALUES (:id, 'ACCEPTED');
INSERT INTO outbox(event_id, aggregate_id, payload)
VALUES (:event_id, :id, :payload);
COMMIT;
-- A separate relay publishes outbox rows later.
```

Verify it: Roll back either insert and confirm both rows disappear. Test duplicate relay publishing and durable consumer deduplication.

8. Saga [Medium; later design round]

Plain English: A workflow across several services is split into local steps. If a later step fails, the system performs a defined compensating action where possible. Compensation is a new business operation, not a magic database rollback.

Professional answer: "I consider a saga when one business process crosses independently owned services and a single database transaction cannot cover it. I define states, retry and compensation behavior, and what happens after an irreversible step. For a simple workflow within one database, a local transaction is safer and easier."

Example: Reserve funds, submit instruction, record confirmation. If submission fails before an irreversible action, release the reservation. If the external outcome is unknown, reconcile before resubmitting. **Cost:** State tracking, delayed consistency, failed compensation and operational complexity. See Microsoft saga pattern.

Demonstration: A cross-service onboarding workflow creates a customer identity, provisions an account, and activates a payment profile. If account provisioning fails, the orchestrator requests an identity cleanup where business rules allow. If profile activation has already caused an irreversible outside action, the workflow needs a new reversal or review step, not a fake rollback.

Why saga: Each service commits to its own store, and no safe single ACID transaction covers the workflow. **Proof:** Test failure at each step, including a failed compensation, and ensure the saga ends in a visible `NEEDS_REVIEW` state instead of silently disappearing. **Avoid it:** One service and one local database transaction need no saga.

Code anchor (illustrative fragment):

JAVA

```java
void handle(ReservationSucceeded e) {
    saga.markReserved(e.orderId());
    clearing.submit(e.orderId());
}

void handle(SubmissionRejected e) {
    reservation.requestRelease(e.orderId());
    saga.markCompensating(e.orderId());
}
// Compensation is another action; it can fail.
```

Verify it: Simulate compensation failure and an unknown external outcome. Do not assume a remote charge can be rolled back.

9. CQRS [Medium; later design round]

Plain English: Commands change state; queries read it. CQRS separates their models so each can fit its job. This can be as small as separate code paths in one database or as large as a separate read store.

Professional answer: "I use CQRS only when read and write needs differ enough to justify separate models. For example, payment changes may enforce strict invariants while reconciliation dashboards need a denormalized view. I would define how the view is updated and what users see during lag."

Cost: More code and consistency management. CQRS does not require event sourcing, a message broker, or microservices. See Microsoft CQRS pattern.

Demonstration: A payment write path enforces approval and settlement invariants using normalized transactional tables. A reconciliation dashboard reads a denormalized projection keyed by tenant/date/status to avoid expensive joins for every chart.

Why CQRS: The write model optimizes correctness; the read model optimizes a genuinely different query shape. **Proof:** Compare dashboard latency and write throughput, and test how the UI displays a projection that is several seconds behind. **Limit:** CQRS can use one database and does not imply event sourcing; if one simple query meets the need, separation adds needless lag and operational work.

Code anchor (illustrative fragment):

JAVA

```java
// Write model owns invariants and state transitions.
void approve(ApprovePayment cmd) { /* validate + persist */ }

// Read model is optimized for a screen.
record PaymentSummary(String id, String status, Money total) {
}

PaymentSummary dashboardRow(PaymentId id) { /* projection */ }
```

Verify it: Measure projection lag when using a separate read store. CQRS can also use one database and separate query models.

10. Event sourcing [Low for this round]

Plain English: Store a sequence of events as the source of truth and rebuild current state from them. An audit log beside a normal database is not automatically event sourcing.

Professional answer: "Event sourcing can provide a detailed history and support rebuilding projections, but it adds schema evolution, replay, ordering, and operational complexity. I would use it only for a clear requirement that outweighs those costs."

Example: Account ledger events may be valuable, but financial audit requirements alone do not force full event sourcing. Know snapshots and replay only at a conceptual level now.

Demonstration: Rather than storing only `account.balance = 500`, an event-sourced ledger stores an ordered history such as `AccountOpened`, `FundsCredited(700)`, and `FundsDebited(200)`; the current balance is derived by replaying the accepted events.

Why event sourcing: Reconstructing prior state and knowing which accepted changes produced it are primary requirements. **Proof:** Rebuild a projection from an empty state and compare it with the live one; test event-version upgrades and replay handling. **Cost:** Corrections become new events, projections lag, and historical schemas must remain interpretable. A table of audit records alongside current state is not event sourcing.

Code anchor (illustrative fragment):

JAVA

```java
record Deposited(Money amount) implements AccountEvent {
}

record Withdrawn(Money amount) implements AccountEvent {
}

Balance replay(List<AccountEvent> events) {
    Balance balance = Balance.zero();
    for (AccountEvent e : events) balance = apply(balance, e);
    return balance;
}
// Current balance derives from stored events.
```

Verify it: Replay event history and compare the final state. Audit logs next to mutable balances are not automatically event sourcing.

11. Cache-aside [High]

Plain English: Read the cache first. On a miss, read the database and fill the cache. When source data changes, invalidate or update the cached copy according to a freshness policy.

Professional answer: "I use cache-aside for repeated reads where stale data is acceptable within a defined bound. I set tenant-aware keys, TTLs, invalidation rules, and stampede protection, then measure hit rate and tail latency. I keep authoritative payment state and permissions on a deliberate consistency path."

Cost: Staleness, extra failure mode, memory, invalidation races. A 35% latency improvement is useful only when the baseline and workload are known. See Redis cache-aside guidance.

Demonstration: A public currency-metadata lookup reads `cache[tenant+currency]`; on a miss it reads the authoritative store and fills the cache. On a metadata update, invalidate the affected key. The key must include every dimension that changes the answer.

Why cache-aside: Repeated reads can be served faster when a defined staleness window is tolerable. **Proof:** Measure hit rate and p95 latency under a representative workload; test concurrent miss storms and invalidation races. **Limit:** Do not treat cached balance, mutable payment status, or authorization as authoritative without a separately justified consistency design.

Code anchor (illustrative fragment):

JAVA

```
PaymentView get(PaymentId id) {
    PaymentView cached = cache.get(id);
    if (cached != null) return cached;
    PaymentView fresh = repository.readView(id);
    cache.put(id, fresh, ttl);
    return fresh;
}
// Writes must invalidate/update this view deliberately.
```

Verify it: Test a cache miss, a hit, and staleness after a write. Use tenant-aware keys and exclude unsafe permission shortcuts.

12. Circuit breaker, retry, timeout, and bulkhead [High]

Plain English: A timeout limits waiting. A retry tries again when failure may be temporary and repetition is safe. A circuit breaker stops calls to a dependency that keeps failing. A bulkhead limits how much of the application's resources that dependency can consume.

Professional answer: "I combine bounded timeouts, selective retries, a circuit breaker, and separate resource limits according to the dependency's failure behavior. I also define a safe fallback or clear error. For payments, a timeout is an uncertain outcome, so idempotency and reconciliation must be part of retry design."

Cost: A retry storm can make an outage worse; a circuit breaker can reject a recovered service until probing succeeds; bulkheads use extra capacity. Tune from latency and error data. See Microsoft circuit breaker and bulkhead.

Demonstration — provider slows down: Set a per-attempt deadline so calls do not wait forever. Retry only safe transient errors with bounded attempts and jitter. If failure rates stay high, open a circuit breaker to fail fast. Give provider calls a separate small concurrency pool so an outage does not exhaust threads needed for status APIs.

Why all four: Timeout bounds a call; retry handles temporary failure; breaker avoids repeated work against an unhealthy dependency; bulkhead limits the blast radius. **Proof:** In a test outage, status queries still work, calls are capped, and request traffic does not grow into a retry storm. **Payment caution:** A timed-out submission has an **unknown** outcome until deduplication/reconciliation proves otherwise.

Code anchor (illustrative fragment):

JAVA

```
// Pseudocode: actual library API and ordering vary.
withBulkhead("clearing",() ->

withCircuitBreaker("clearing",() ->

retryOnlyIfSafe(() ->

callWithDeadline(provider::submit))));
// A timeout does not prove whether provider acted.
```

Verify it: Inject high latency and persistent failures. Verify resource isolation and bounded attempts; prevent duplicate charges.

13. Queue-based load leveling [Medium]

Plain English: Put work into a durable queue when producers can send faster than consumers can safely process. Consumers take work at a controlled rate.

Professional answer: "I use a queue when the caller does not require immediate completion and burst traffic would overload a dependency. I define capacity, acceptance response, retry, duplicate handling, dead-letter treatment, and backlog monitoring."

Cost: Delayed results, eventual consistency, operational queues, ordering limitations. A queue does not remove a bottleneck; it changes how you absorb and expose it. See Microsoft load-leveling pattern.

Demonstration: A partner sends a burst of 30,000 import records, while the clearing integration can process only a smaller controlled rate. The API durably enqueues validated work and returns a job ID; consumers drain the queue at the downstream-safe rate.

Why queueing: It absorbs bursts without requiring the slower dependency to match peak arrival rate. **Proof:** During the burst, queue depth and oldest-message age rise predictably and later return to normal; duplicate deliveries do not duplicate payment effects. **Cost:** The queue does not increase steady-state provider capacity. Set backlog limits, retention, poison-message routing, and an honest `PENDING` status.

Code anchor (illustrative fragment):

JAVA

```java
@PostMapping("/reports")
ResponseEntity<?> accept(ReportRequest request) {
    JobId id = jobs.persist(request); // durable first
    queue.enqueue(id);
    return ResponseEntity.accepted().body(id);
}
// Worker processes at bounded concurrency.
```

Verify it: Burst-load the endpoint and confirm stable worker concurrency. Handle the DB-plus-queue dual write with an outbox or equivalent.

14. Strangler Fig migration [Medium]

Plain English: Replace a legacy system a piece at a time. Route selected capabilities through new code while the old system continues handling the rest.

Professional answer: "For a risky modernization, I favor a controlled migration by capability or customer segment. I define routing, data ownership, reconciliation between old and new behavior, rollback, and a retirement plan for the legacy path."

Example: Move one payment type or client group from file-based processing to a new service, compare results, expand traffic gradually, then remove the old path. **Cost:** Temporary duplication and complex coexistence. See Martin Fowler's Strangler Fig pattern.

Demonstration: During a file-to-API migration, route one well-defined payment category to the new path, keep the others on the legacy path, and compare submitted totals and reconciliation outcomes by route. Increase traffic only after parity and failure criteria are met.

Why Strangler Fig: It replaces a capability incrementally instead of risking a full cutover. **Proof:** A routing flag can redirect new eligible work to the legacy path without moving already-processing transactions; reconciliation finds no missing or double-submitted instructions. **Tradeoff:** Temporary dual operations, schema translation, observability, and a clear retirement plan are necessary.

Code anchor (illustrative fragment):

JAVA

```java
PaymentResult submit(Payment p) {
    if (routing.usesNewProvider(p.customerId()))
        return newGateway.submit(p);
    return legacyGateway.submit(p);
}
// Route each instruction once; preserve its chosen path.
```

Verify it: Compare reconciliation outcomes by cohort. Rollback routing for new work only; avoid submitting an in-flight instruction twice.

5. Worked design: submit a commercial payment

This is a **practice scenario**, not a claim about your employer's implementation. The interviewer asks: "Design the backend for submitting a commercial payment instruction. A customer might retry after a timeout. Downstream processing is asynchronous."

Step 1: Define the contract

Ask: What is the final business outcome? Is the API acknowledging receipt or settlement? How are tenant, amount, currency, beneficiary and authorization validated? What throughput and response-time goal matters? How long should duplicate keys be remembered? Which downstream outcome can be uncertain?

Proposed contract: `POST /payment-instructions` accepts a tenant-scoped idempotency key and a validated request. A successful response means **accepted for processing**, not settled. Return a durable instruction ID and a status resource. A repeated same-key/same-payload request returns the recorded outcome; same key with a different payload is a conflict.

Step 2: Draw the main flow

EXAMPLE

```
Client
  -> API: authenticate; authorize tenant and account; validate request
  -> Application service: check idempotency and business rules
  -> Local database transaction: instruction + key/result + outbox event
  -> Response: accepted instruction ID and current status

Outbox relay -> durable broker -> payment processor -> provider
                                        |                  |
                                        +-> status update <-+
Status API -> authoritative instruction state
```

The core service depends on a payment store and an external gateway **port**. The REST controller, database repository, outbox relay, and provider client are **adapters**. The domain model protects status transitions. A local transaction protects instruction, deduplication record and outbox record. A separate worker handles the external call.

Step 3: Walk through failures

Failure 1: Client retries after response was lost

Expected behavior: Return the original instruction for the same tenant, key, and normalized payload.

Proof / test: Crash immediately after commit, then retry: ID stays the same; no second charge.

Failure 2: Two identical requests arrive together

Expected behavior: Use a durable uniqueness constraint, not just check-then-insert.

Proof / test: Send parallel requests from separate instances: one instruction and one outbox fact.

Failure 3: Database commit fails

Expected behavior: Roll back instruction, idempotency record, and outbox together.

Proof / test: Force a DB failure: none of the rows remain and acceptance is not returned.

Failure 4: Relay publishes twice

Expected behavior: Make consumer effects durable and idempotent by event/business key.

Proof / test: Deliver the event twice: one applied business effect, duplicate observed.

Failure 5: Provider times out

Expected behavior: Treat the outcome as unknown; query status or reconcile before resubmitting.

Proof / test: Simulate a provider action followed by lost response: no blind second submission.

Failure 6: Downstream backlog grows

Expected behavior: Bound worker concurrency; expose PENDING; alert on oldest-message age.

Proof / test: Burst-load: work stays bounded; queue depth and oldest age are visible.

Failure 7: Permission is revoked

Expected behavior: Recheck authorization for each new operation; do not trust stale role data.

Proof / test: Try cross-tenant access and a revoked role: both are denied.

Step 3A: A transaction boundary in code

JAVA

```java
@Service
final class AcceptInstruction {
    private final InstructionRepository instructions;
    private final OutboxRepository outbox;

    @Transactional
    public Receipt accept(Instruction instruction) {
        // Unique constraint on (tenant_id, idempotency_key).
        Receipt receipt = instructions.insertOnce(instruction);
        outbox.insert(PaymentAccepted.of(receipt.id()));
        return receipt; // This means ACCEPTED, not SETTLED.
    }
}
```

Why this is not the whole solution: The actual duplicate-key conflict must return the previously recorded response only after verifying a matching payload fingerprint. The relay and consumers must be safe for repeated delivery. The remote clearing provider is outside this database transaction. Test the concurrent same-key case and the rollback case with a real database.

> NOTE: DON'T PRINT BELOW TABLE WHEN CREATING PDF

Failure	Expected behavior	Check to make
Client retries after response was lost	Same key and payload find the existing instruction	No second instruction or charge
Two identical requests arrive together	Durable uniqueness makes one the winner	Concurrent test across instances or transactions
Database commit fails	No accepted instruction or outbox event	Rollback and error response
Relay publishes twice	Consumer recognizes duplicate event	One business effect; observable duplicate count

Failure	Expected behavior	Check to make
Provider times out	Outcome remains uncertain until queried or reconciled	Do not blindly submit again
Downstream backlog grows	Limit work and expose pending status; alert on age	Queue age, depth, error rate
Permission is revoked	Each new operation checks current authorization policy	Cross-tenant and revoked-role tests

Step 4: Explain tradeoffs

Synchronous provider submission could give faster final feedback, but lengthens request latency and couples availability to the provider. Asynchronous submission gives a quick durable acceptance, but customers need a status view and understand pending outcomes. A local database transaction is enough for local state and the outbox; it cannot atomically include the remote provider. The design needs provider-side idempotency or reconciliation for uncertain outcomes.

Polished 90-second answer: "I would first separate accepting the instruction from final processing. The API authenticates the caller, verifies tenant and account access, and validates the request. I would require a tenant-scoped idempotency key and store the instruction, deduplication result, and outbox event in one local transaction. The response would say 'accepted' with an instruction ID. A worker publishes and processes the event, with duplicate-safe handling and monitored retries. The hardest case is a provider timeout: I would treat that as an uncertain outcome, check the provider using a stable reference or reconcile it, and avoid a blind second charge. I would measure pending age, error rate, duplicate attempts, and final outcomes. This adds asynchronous workflow complexity, but it isolates the API from provider latency and gives a clear recovery path."

Step 5: Tie back to your real experience

Your resumes discuss payment modernization, Java APIs, identity, caching, reliability, and reconciliation. Choose **one** real project and fill in: Which parts of this practice design did you personally build? Which parts were already present? Which failure actually occurred? Which metric can you verify? Do not present this example system as one you implemented unless it matches your work.

6. Fast comparisons interviewers ask

These vertical comparisons avoid landscape-only tables. Use the final question as your interview decision rule.

Comparison 1: Strategy vs State

First concept: Strategy selects interchangeable rules, such as domestic and cross-border fees.

Second concept: State limits behavior based on current lifecycle status, such as CREATED or SETTLED.

Decision question: Am I choosing a rule, or validating a state transition?

Comparison 2: Adapter vs Facade

First concept: Adapter translates a legacy provider protocol into the application contract.

Second concept: Facade presents one use-case entry point to several collaborators.

Decision question: Am I translating an incompatible interface, or simplifying orchestration?

Comparison 3: Decorator vs Proxy

First concept: Decorator adds composable behavior, such as measuring call latency.

Second concept: Proxy controls access or lifecycle, such as Spring transaction advice.

Decision question: What purpose does the wrapper serve?

Comparison 4: Factory vs Builder

First concept: Factory chooses or creates a concrete handler.

Second concept: Builder assembles one complex request with optional parts.

Decision question: Is selection the problem, or safe construction?

Comparison 5: Layered vs Hexagonal

First concept: Layered assigns controller, business, and persistence responsibilities.

Second concept: Hexagonal requires the core to own ports, with technology at the edges.

Decision question: Do infrastructure dependencies leak into the core?

Comparison 6: Modular monolith vs microservices

First concept: Modular monolith has one deployable with internal capability boundaries.

Second concept: Microservices have independent deployments and explicit network/data boundaries.

Decision question: Will independent operations outweigh distributed complexity?

Comparison 7: Event vs command

First concept: Event states a completed fact: PaymentApproved.

Second concept: Command requests an action: ApprovePayment.

Decision question: Has the business change happened already?

Comparison 8: Local transaction vs saga

First concept: Local transaction gives atomic changes within its database scope.

Second concept: Saga coordinates independently committed steps and compensations.

Decision question: Can one transaction own all affected state?

Comparison 9: CQRS vs event sourcing

First concept: CQRS separates read and write models.

Second concept: Event sourcing keeps events as the primary durable state history.

Decision question: Is the need separate query shape, or event replay as source of truth?

Comparison 10: Retry vs circuit breaker

First concept: Retry repeats a safe operation after a transient failure.

Second concept: Circuit breaker stops calls when continued failures are likely.

Decision question: Is an extra attempt safe and likely to help?

Comparison 11: Bulkhead vs rate limit

First concept: Bulkhead isolates internal concurrency/resource pools.

Second concept: Rate limit restricts incoming request volume or quota.

Decision question: Which boundary needs protection?

Comparison 12: Cache vs read model

First concept: Cache stores a faster copy of the same data.

Second concept: Read model reshapes data for a specific query/use case.

Decision question: Is the need faster access or a different view?

> NOTE: DON'T PRINT BELOW TABLE WHEN CREATING PDF

Choice	First option	Second option	Decision question
Strategy vs State	Selects a policy	Models state-dependent behavior and transitions	Does behavior vary by selected rule or by lifecycle state?
Adapter vs Facade	Translates an incompatible interface	Simplifies access to a complex subsystem	Are you changing the interface or hiding orchestration?
Decorator vs Proxy	Adds composable behavior	Controls access or lifecycle	What is the wrapper's intent and effect?

Choice	First option	Second option	Decision question
Factory vs Builder	Selects or creates an implementation	Constructs a complex value step by step	Is the problem which object to create or how to assemble one?
Layered vs Hexagonal	Separates code by responsibility	Makes core policy depend on ports rather than infrastructure	Are external dependencies changing or isolated tests important?
Modular monolith vs microservices	One deployment with modules	Independent deployments and owned data	Do team autonomy and scaling needs justify distributed costs?
Event vs command	Describes a completed fact	Requests an action	Has the change happened yet?
Local transaction vs saga	Atomic within one database boundary	Coordinates local steps across services	How many owners and data stores must change?
CQRS vs event sourcing	Separates reads and writes	Stores events as the source of truth	Are you optimizing read/write models or preserving all changes for replay?
Retry vs circuit breaker	Repeats a safe transient operation	Temporarily stops calls likely to fail	Is recovery likely soon, or is persistent failure harming the system?
Bulkhead vs rate limit	Isolates internal resource pools	Restricts incoming usage	Where must overload be contained?
Cache vs read model	Copies data for faster access	Builds a purpose-specific projection	Is the need speed for the same value or a different query shape?

7. Practice questions and model answers

Question 1: "Explain SOLID without reciting the letters."

Answer: "SOLID helps me keep responsibilities clear, define reliable contracts, and control dependencies. For example, if I add a new payment route, I would not want to modify one giant class that handles validation, fees, storage, and provider calls. I would separate stable orchestration from the parts that vary. I use those principles when they make changes and tests safer, rather than creating abstractions everywhere."

Question 2: "Why would you choose Strategy over a switch statement?"

Answer: "If there are only two simple fixed cases, a switch can be clearer. I move to Strategy when policies grow, change independently, or need separate tests. Then each policy implements one contract and the caller does not contain every rule. I would also make the selection rule explicit so it does not become a hidden second switch."

Question 3: "How is Hexagonal different from layering?"

Answer: "Both can separate responsibilities. Hexagonal makes the dependency direction more explicit: business code defines the ports it needs, and database or API adapters implement them. That is useful when integrations change or business logic needs isolated tests. Layering alone does not guarantee that the business layer is free from infrastructure details."

Question 4: "When would you choose a modular monolith over microservices?"

Answer: "I would start with strong modules if the workflow is cohesive and one team can deploy it safely. A modular monolith keeps local transactions and debugging simpler. I would split services when independent ownership, release cadence, or scaling has enough value to justify network calls, data consistency, monitoring, and operational overhead."

Question 5: "Does the outbox guarantee exactly-once processing?"

Answer: "No. It protects the local database change and event record from a dual-write gap. The relay may publish a message more than once, and a consumer may retry. I would make the consumer idempotent using a stable event ID or business key and check the effect in durable storage."

Question 6: "Can you roll back a saga?"

Answer: "A saga uses compensating business actions after local transactions have committed. It is not a single database rollback. Some effects, such as an external payment already settled, may be irreversible; the right response could be a reversal or manual reconciliation. I would define that behavior before relying on the saga."

Question 7: "What if the provider call times out?"

Answer: "A timeout tells me our response did not arrive within the deadline; it does not tell me whether the provider acted. I would record an uncertain state and query or reconcile using a stable reference. I would retry only under the provider's idempotency contract and the business rule for duplicate prevention."

Question 8: "Which pattern did you use in your last project?"

Answer template: "The problem was [actual problem]. I used [pattern or design] by [specific implementation I owned]. I considered [alternative]. The tradeoff was [real cost], and I verified it with [test or production measure]." If a formal pattern name does not fit, describe the design without forcing a label.

Rapid oral drill

Give each answer in 30-45 seconds, then add one tradeoff: SRP versus tiny classes; DIP versus dependency injection; Strategy versus State; Adapter versus Facade; Repository versus generic CRUD wrapper; modular monolith versus microservices; command versus event; outbox versus saga; CQRS versus event sourcing; retry versus circuit breaker; timeout versus cancellation; bulkhead versus rate limit; cache invalidation; strangler migration rollback; one pattern you personally used.

8. Four worked demonstrations: prove the choice, not just the name

These are **illustrative Java 17+/Spring-style examples**, not complete payment code. They omit authentication plumbing, currency precision rules, full persistence mapping, and vendor SDK contracts. They are practice designs, **not implementations attributed to a real employer**.

Demonstration A: change a fee rule without changing submission orchestration

Starting problem: One `submit()` method contains fee switches, builds external requests, and saves payment rows. Adding a fee policy means editing and retesting unrelated provider code.

JAVA

```java
import java.math.BigDecimal;

interface FeePolicy {
    BigDecimal calculate(PaymentDraft payment);
}

record PaymentDraft(String tenantId, BigDecimal amount, String currency) {
}

final class FixedFeePolicy implements FeePolicy {
    public BigDecimal calculate(PaymentDraft payment) {
        return new BigDecimal("1.50"); // Illustrative rule only.
    }
}

interface PaymentStore {
    void save(PaymentDraft payment, BigDecimal fee);
}

final class SubmitPaymentUseCase {
    private final FeePolicy feePolicy;
    private final PaymentStore store;

    SubmitPaymentUseCase(FeePolicy feePolicy, PaymentStore store) {
        this.feePolicy = feePolicy;
        this.store = store;
    }

    void accept(PaymentDraft payment) {
        BigDecimal fee = feePolicy.calculate(payment);
        store.save(payment, fee);
    }
}
```

Reasoning: SRP gives fee decisions and persistence separate reasons to change. Strategy defines interchangeable fee rules; composition injects the choice. DIP makes the use case depend on `PaymentStore`, not JDBC. **Evidence to show in an interview:** A new fee-policy implementation is tested without changing `SubmitPaymentUseCase`; an in-memory store validates what the use case passes to persistence. **Tradeoff:** This

adds interfaces; for a single fixed fee rule in a short-lived tool, direct code may be simpler. **Not production ready:** Currency-specific scale/rounding, validation, authorization, audit, idempotency and transactions remain to be designed.

Demonstration B: protect accepted payments from a lost response

Scenario: Two clients retry the same instruction while the first API response is lost. A process-local `synchronized` block is not enough when multiple instances accept requests.

EXAMPLE

```
Request (tenantId, idempotencyKey, normalizedPayload)
     |
     v
Single local DB transaction
     +-- insert instruction with durable ID
     +-- insert idempotency record (UNIQUE tenantId, key)
     +-- insert outbox record (UNIQUE eventId)
     |
     v
Commit -> return instruction ID + ACCEPTED
             |
             v
          Relay -> broker -> deduplicating consumer
```

Detailed decision: Store a stable payload fingerprint with the key. A repeat with the same key and equivalent payload returns the previously recorded instruction; a repeat with a different payload is a conflict. Let a **database uniqueness constraint** decide simultaneous inserts, then read the winner in a new, valid transaction after a uniqueness conflict. Do not rely on `check-then-insert` without a unique constraint. Agree on key retention with the business.

How to prove it: Run two concurrent requests from different instances with one `(tenant, key)`; verify exactly one instruction and outbox record. Crash after commit but before HTTP response; a repeat returns the same ID. Crash after publish but before relay checkpoint; a duplicate event is possible but consumer effects must be idempotent. **Tradeoff:** Durable dedup records and relay monitoring are operational costs. This does not prevent a second provider charge unless the downstream submission path also uses safe provider-side idempotency or reconciliation.

Demonstration C: distinguish Strategy, State, Command, and Event

A single payments workflow can use these patterns for **different questions**:

How is the fee chosen?: Strategy: choose CrossBorderFeePolicy; test correct policy selection.

What may happen next?: State: SETTLED cannot return to CREATED; test invalid transitions.

What action is requested?: Command: ApprovePayment (id, actor); test unauthorized commands have no effect.

What already happened?: Event: PaymentApproved (id, time); test duplicate consumers are safe.

Why this demonstrates understanding: A strategy does not express payment lifecycle, and an event should not be emitted as if approval happened before the approval transaction commits. Using one word—"event"—for both a request and a completed fact hides critical failure handling.

Demonstration D: choose the smallest correct architecture

One team, one DB: Start with a layered modular monolith. Split only if independent deployments or scaling justify the cost.

Changing clearing providers: Use a port and adapter inside the service. A new service deployment is not necessary.

DB change must cause a broker event: Use a transactional outbox; a saga is not a substitute for atomic local writes.

Cross-service compensations: Use a saga with explicit state only when multiple independent services own the steps.

Dashboard reads differ from writes: Measure query/index limits before adding a separate CQRS projection.

Defend the choice: State the ownership boundary, consistency guarantee, worst failure case, measurable requirement, and what complexity you decline to introduce. "Microservices scale better" is not a sufficient justification by itself.

Five-minute self-check: answer with a counterexample

1. A new payment channel appears: why might a switch still be better than Strategy plus Factory?
2. A subclass compiles but throws for a documented parent operation: which promise was violated?
3. A Spring method calls its own `@Transactional` method: what boundary was bypassed?
4. An outbox relay publishes one event twice: where is the duplicate business effect prevented?
5. A provider submission times out: why is retry not automatically safe?
6. A reconciliation dashboard is slow: why test indexes/queries before adopting CQRS?

Suggested answer shape for each: "The observed problem is __. This mechanism helps because __. A simpler alternative is __. I would prove the need with __. Its failure mode is __." A good answer can also conclude that **no pattern change is justified yet**.

Answer guide: (1) Use a switch when variation is small and stable; abstraction costs more than it saves. (2) LSP: the subtype broke its promised behavior. (3) A self-call bypassed Spring proxy advice in standard proxy mode. (4) At the consumer, using a durable event ID or business key and an atomic effect/deduplication record. (5) The provider may have committed the payment even though the response was lost. (6) CQRS adds projection updates and consistency lag, so first measure whether simpler queries meet the requirement.

Runnable Java drills (no extra libraries)

Three self-contained Java 17+ files are included in the `examples/` folder, so you can see a passing behavior rather than only reading snippets:

- `FeeStrategyDemo.java`: replace the fee policy without changing the processor (Strategy, SRP, composition).
- `PaymentStateDemo.java`: permit only valid payment transitions (encapsulation, State).
- `IdempotencyToyDemo.java`: return one instruction ID for a repeated same-key request and reject a changed payload. **This deliberately uses in-memory storage and is not sufficient for a real distributed payment API.**

Run from the `examples` folder:

BASH

```
javac FeeStrategyDemo.java PaymentStateDemo.java IdempotencyToyDemo.java
java FeeStrategyDemo
java PaymentStateDemo
java IdempotencyToyDemo
```

All three were compiled and executed successfully during preparation. Use the source files to change one input, break one assertion, and explain which contract failed.

Official references for implementation-sensitive claims

- Spring: `@Transactional` and proxy limitations
- AWS: Hexagonal architecture / ports and adapters
- AWS: Transactional outbox
- Microsoft: Saga
- Microsoft: Cache-Aside
- Microsoft: Circuit Breaker

These links support the architectural mechanisms described above. The practice-specific business rules and designs are teaching examples, not facts about any employer or vendor.

9. Final one-page recall

- **Name the problem before the pattern.** A pattern is a tool, not the goal.
- **Principles:** cohesive responsibility; reliable contracts; business policy independent from infrastructure; protect invariants.
- **Simple object patterns:** Strategy varies a rule; Factory selects creation; Builder assembles a complex value; Adapter translates; Facade simplifies; State protects lifecycle transitions.

- **Architecture:** layers separate roles; hexagonal protects the core; modular monolith keeps modules in one deployment; microservices add independent ownership and distributed costs.
- **Data:** local transactions protect one boundary; outbox connects a committed change to an event; saga coordinates local steps; CQRS separates read and write models.
- **Failures:** deadlines, retry safety, circuit breakers, bulkheads, idempotency, reconciliation, and monitoring must fit together.
- **Professional answer:** problem → choice → implementation → tradeoff → failure handling → evidence.
- **Honesty:** "I implemented __; my team used __; I understand __ conceptually." These are different claims.

Two questions for your interviewer: "How does the team decide service boundaries and data ownership?" and "Which failure mode has been most challenging in your payment or servicing workflows?"

Reference scope: Official documentation linked near the relevant architecture patterns was consulted on October 22,

1. These patterns are general practice, not a claim about an American Express internal design. The interview examples are tailored to the experience described in the four resume versions you supplied.